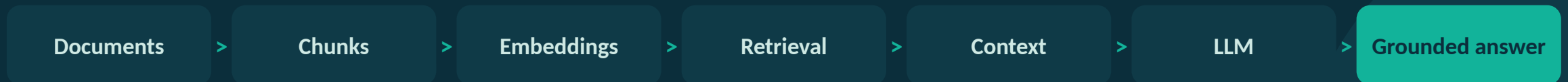


A HANDS-ON TECHNICAL SESSION

# RAG Fundamentals

## From Documents to Grounded Answers

Build a minimal RAG pipeline, step by step — and learn when it should refuse to answer.



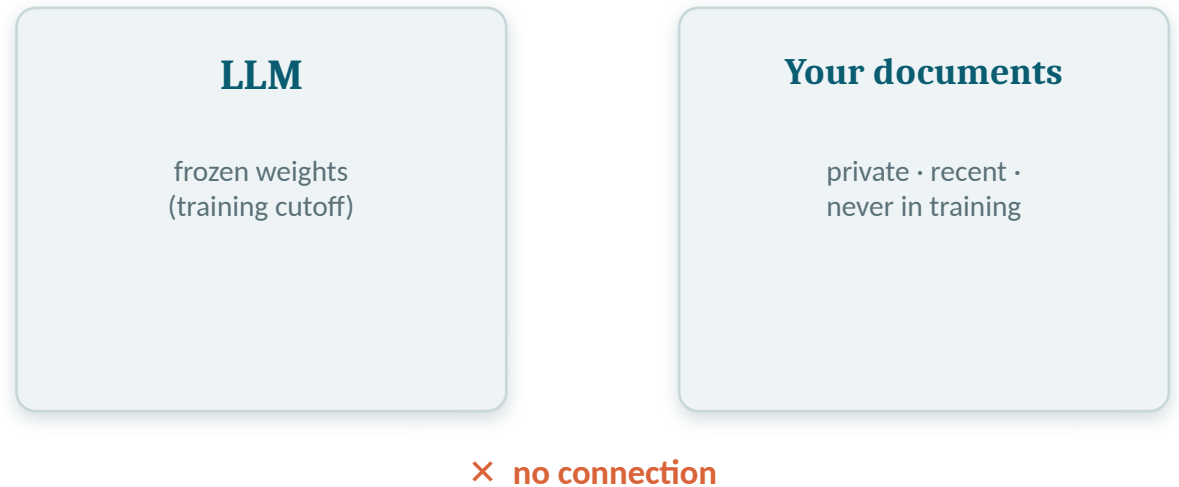
# What you'll be able to do

- Explain why LLMs need retrieval — and what RAG does not fix
- Describe the end-to-end RAG pipeline and name each stage
- Build it: chunk → embed → retrieve Top-K → context
- Generate a grounded, cited answer — and make it refuse
- Recognise the common ways RAG fails

- 1 **Explain**
- 2 **Describe**
- 3 **Build**
- 4 **Ground**
- 5 **Diagnose**

# The problem: an LLM only knows its training data

- A fixed snapshot, frozen at its training cutoff
- Never seen your private docs, wiki, or last week's notes
- Knowledge is parametric — baked into weights, not looked up
- Can't cite it, can't update it, can't audit it



# Hallucination: confident, fluent, and wrong

- Lacking a fact, the model generates a plausible one anyway
- Fluent and confident — which makes errors harder to catch
- Root cause: it predicts likely text, it doesn't consult a source
- High-stakes domains can't tolerate confident fiction

*“According to the 2024 Handbook, section 4.2, the limit is 500 mg twice daily.”*

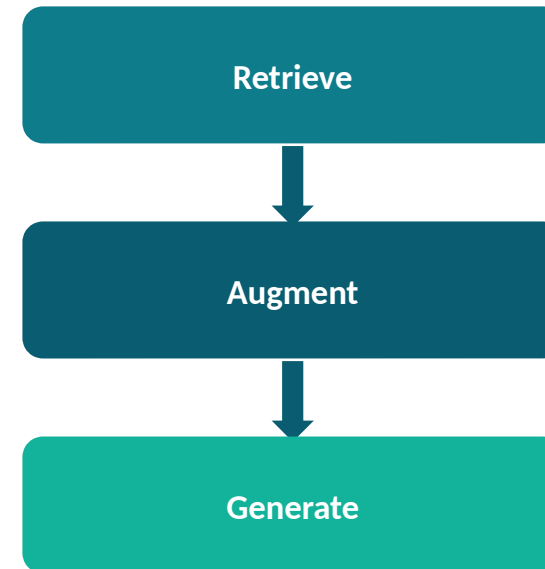
UNVERIFIED

**Fluent. Confident. Ungrounded.**

# What RAG solves

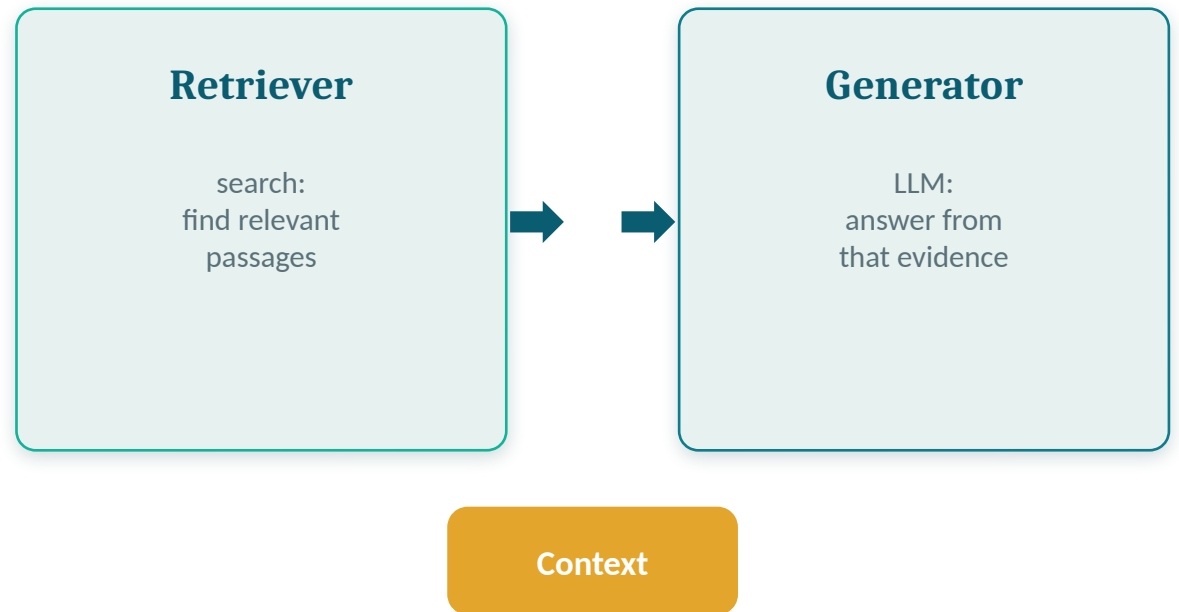
- Retrieve relevant text from your documents at question time
- Augment the prompt with that text as evidence
- Generate an answer grounded in the evidence — and cite it
- Fresh/private knowledge, attribution, and a basis to refuse

The recipe



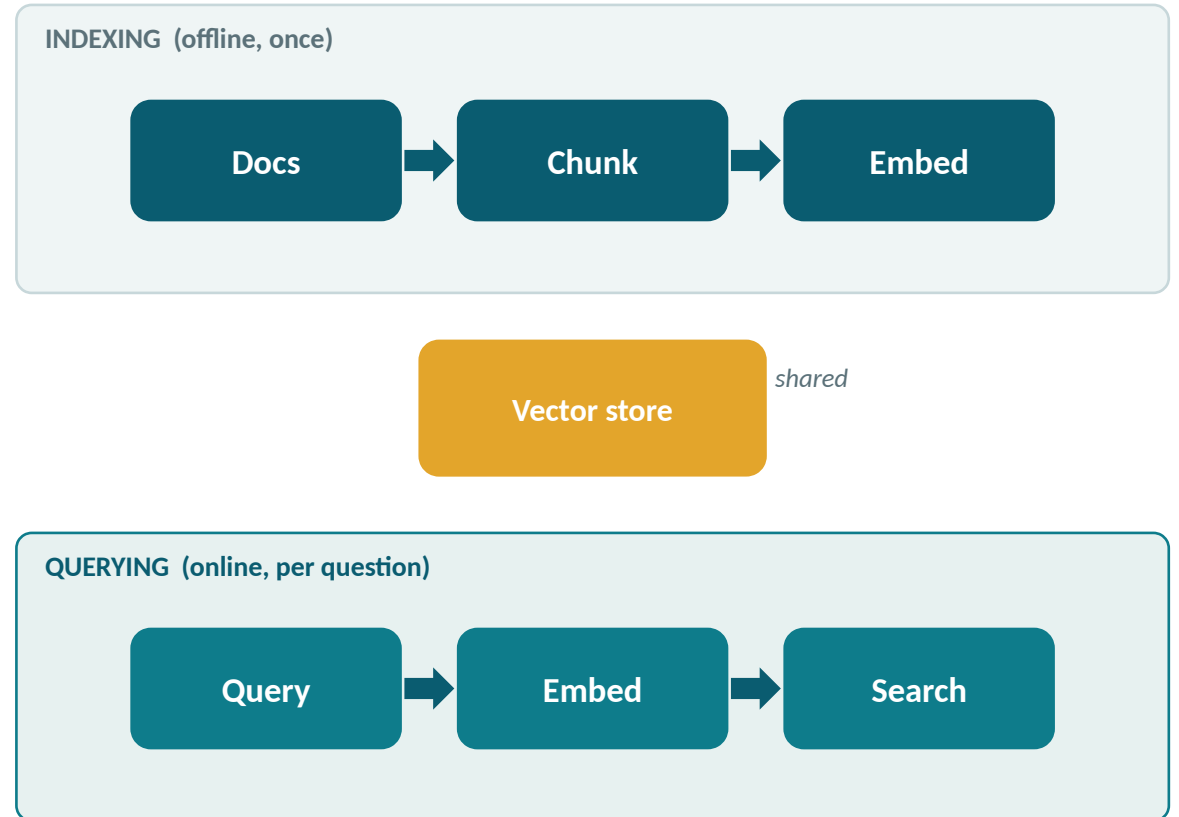
# What is Retrieval-Augmented Generation?

- A retriever that finds relevant passages for a query
- A generator (the LLM) that answers using those passages
- A bridge in the middle: context construction
- The model's job: recall → reading comprehension over evidence



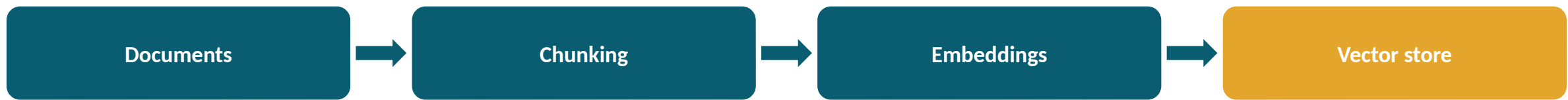
# End-to-end architecture: two phases

- Indexing (offline, once per doc): Documents → Chunk → Embed → Store
- Querying (online, per question): Query → Embed → Search → ... → Answer
- The two phases share one thing: the same embedding model
- Indexing is batch and slow; querying is interactive and fast



# The RAG pipeline (the one diagram to remember)

INDEXING (offline)



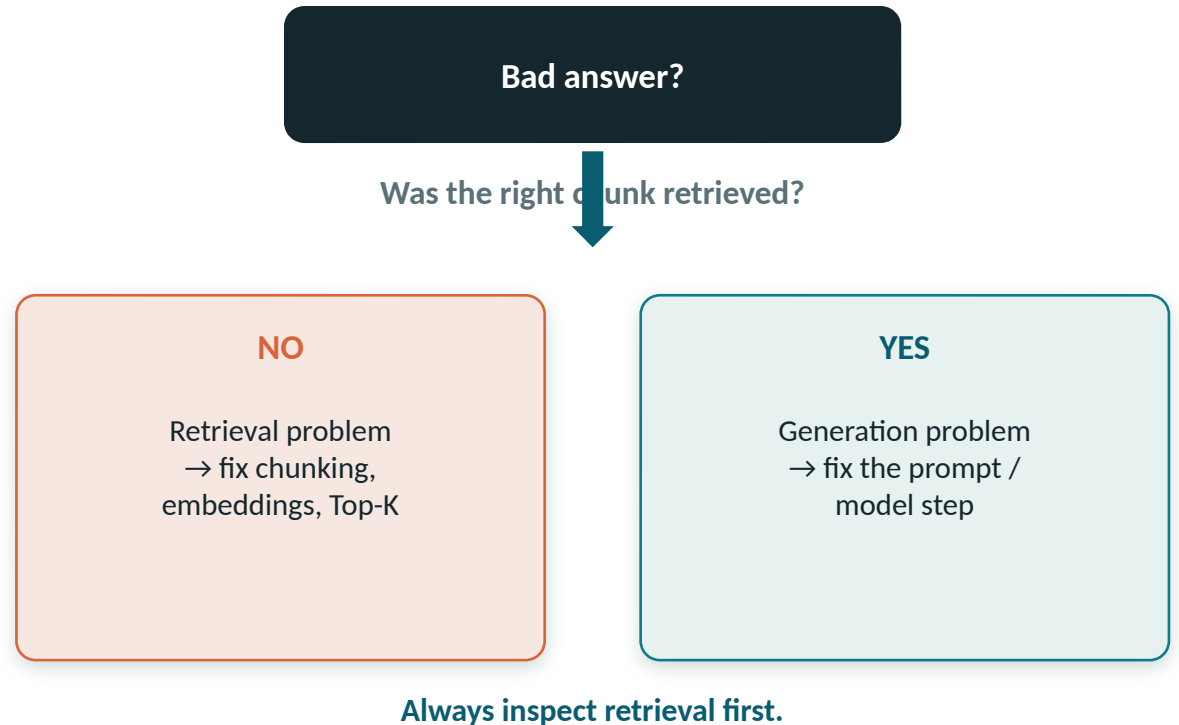
QUERYING (online)



*Every box becomes a function in the lab.*

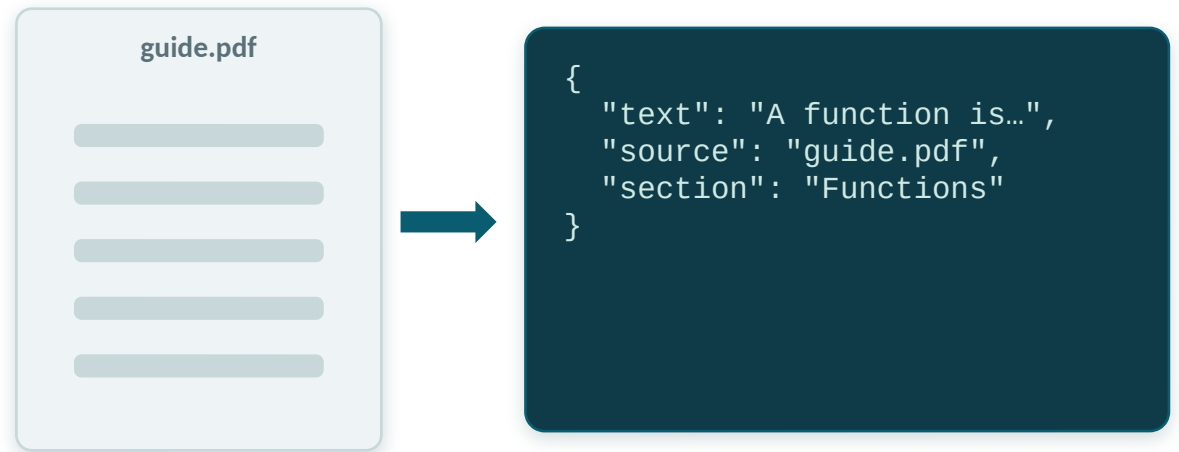
# Retrieval vs Generation: whose fault is a bad answer?

- Retrieval failure: the right passage was never handed over
- Generation failure: it was there, but the model misused it
- Debugging rule: always inspect what was retrieved first
- Most 'the LLM hallucinated' bugs are retrieval bugs



# Documents and preprocessing

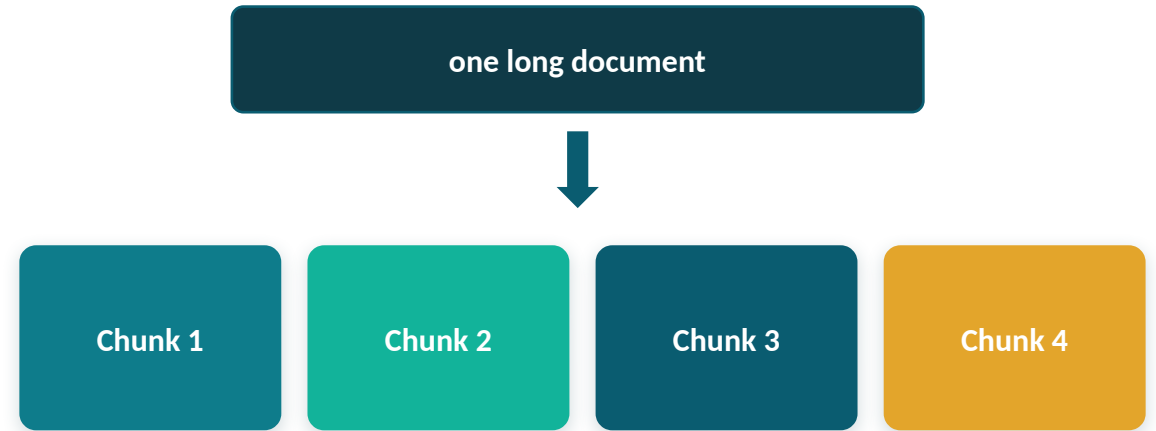
- A document = text + metadata (source, section, page)
- Extract clean text; normalise whitespace
- Keep the metadata — it becomes your citation
- Don't rewrite content: preserve wording, numbers, negations



Keep the metadata → it becomes your citation.

# Chunking: why we cut documents up

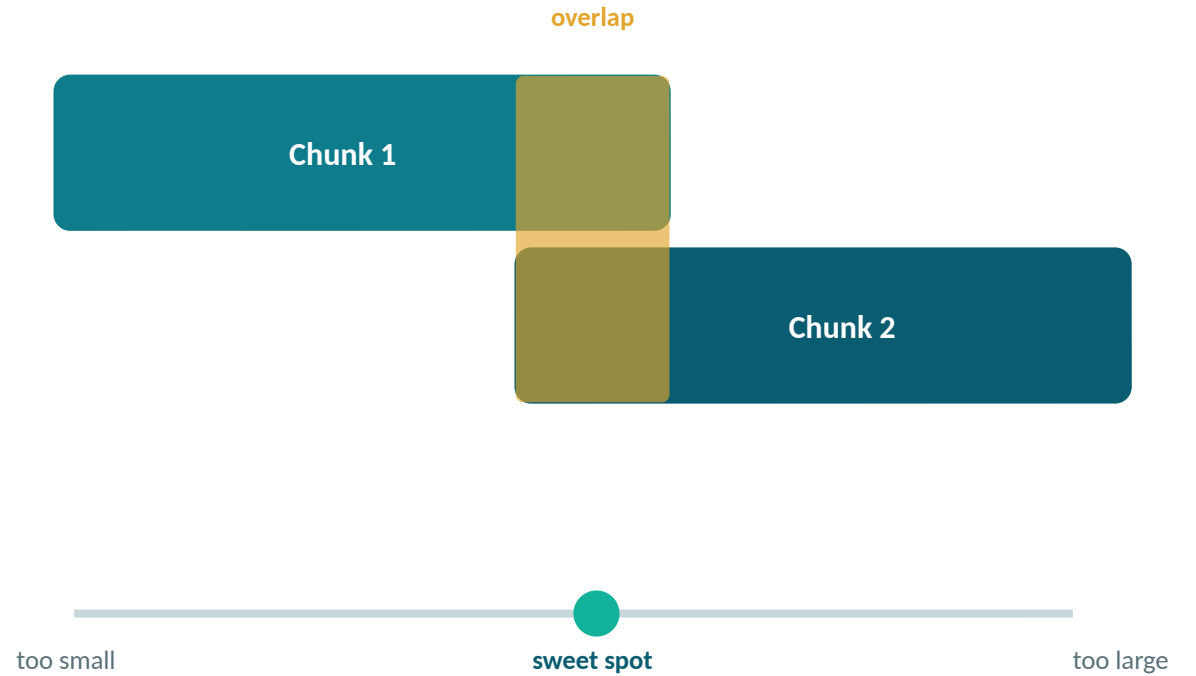
- Embeddings represent a bounded amount of text
- One vector per huge doc = a blurry average of many topics
- Split into chunks: focused, retrievable units
- Chunk on natural boundaries, not mid-sentence



Cut on headings / paragraphs — one chunk, one idea.

# Chunk size and overlap

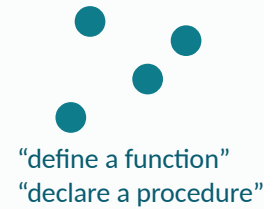
- Too large: many topics → noisy embedding, imprecise retrieval
- Too small: the chunk loses the context that made it meaningful
- Overlap: repeat a few tokens so boundary facts survive
- Start ~ a few hundred tokens, ~10–15% overlap (prod 512/64)



# Embeddings: turning meaning into numbers

- Map text → a fixed-length vector (e.g. 384 numbers)
- Similar meaning → nearby vectors, even with no shared words
- We use all-MiniLM-L6-v2 (local, CPU, no key)
- Embed docs once and each query live — same model

*Similar meaning → nearby points*



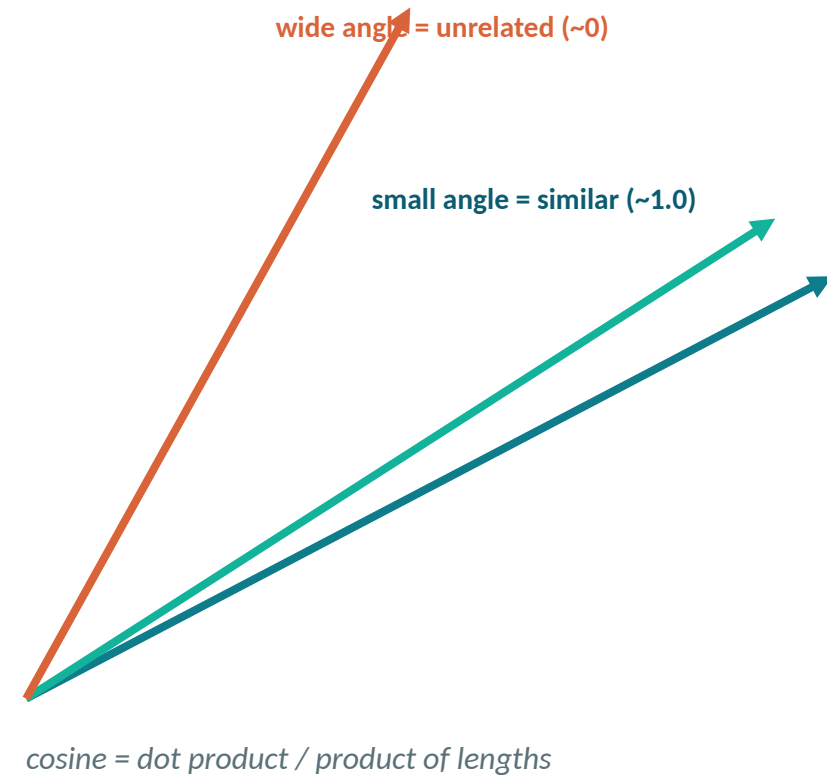
“define a function”  
“declare a procedure”



“HTTP status codes”

# Vector similarity: cosine

- Compare vectors by the angle between them
- 1.0 = same direction (similar); ~0 = unrelated
- Ignores length; measures direction (meaning)
- Just a dot product of normalised vectors — cheap



# Vector search and Top-K retrieval

- Embed query → score all chunks → sort → keep best K
- top\_k too small misses evidence; too large adds noise
- At scale, an ANN index (pgvector/HNSW) replaces brute force
- Production fuses vector + keyword search, then reranks

**0.42** lumen\_cli.md ▶ sync

**0.28** python ▶ functions

**0.19** http ▶ status

0.09 git ▶ commits

0.03 data ▶ tuples

Top-K = 3 (highlighted)

# Building the context for the LLM

- Assemble retrieved chunks into a context block, each tagged with its source
- Keep separate: instructions • evidence • question
- The model sees only this context — missed = gone
- Separation also defends against prompt injection

## SYSTEM

answer only from evidence; refuse if absent; cite sources

## EVIDENCE

[1] (source, section) retrieved chunk text...

## QUESTION

the user's question

# Prompting with retrieved context

- Instruct: answer only from the evidence; invent nothing
- If unsupported, say so — don't guess
- Cite the sources used
- Extractive (quote) vs abstractive (rephrase)

## Ungrounded

- free-form prose
- no sources
- may invent facts
- can't be checked

## Grounded

- quotes evidence
- lists sources
- refuses if unsupported
- auditable

# Citations: provenance, not proof

- Every grounded answer names where it came from
- Citations make answers auditable
- But retrieval is not verification: similar, not necessarily true
- Mantra: retrieved evidence → grounded generation

## Answer

Version 2.3 introduced the --fast flag...

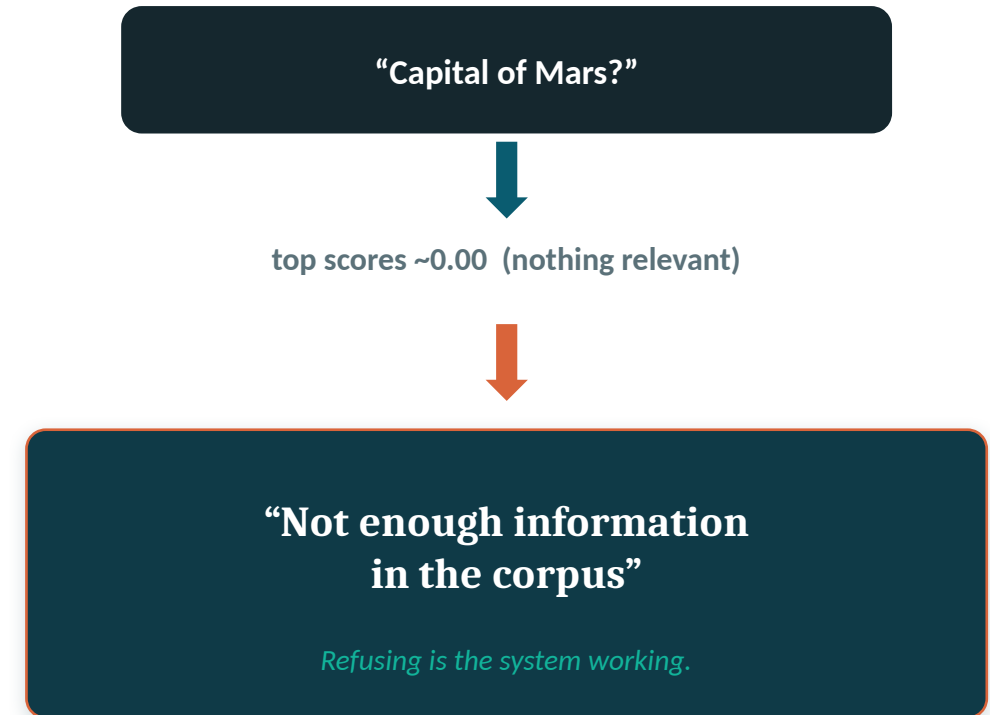
Sources: - lumen\_cli.md > The sync command



provenance, not proof — verify the source

# When retrieval fails: refuse, don't hallucinate

- Retrieval can return nothing relevant (or only weak matches)
- A trustworthy system then refuses
- Implement a grounding guard: a minimum similarity threshold
- Refusing correctly is a feature, not a failure



# Hands-on lab: build a minimal RAG system

- Open rag\_fundamentals\_lab.ipynb — 5 synthetic docs, no key
- Build in order: load → chunk → embed → retrieve → context → generate → cite → refuse
- Rhythm: concept → code → output → 'what did we learn?'
- Stretch: Top-K, chunk size, grounding guard



## RECAP

# What you built — and five traps to avoid

**RAG = Retrieval + Augmented context + Generation**

- 1 Docs aren't "known" — retrieved into context
- 2 More chunks are not better — noise grows too
- 3 Search can fail at every stage
- 4 RAG reduces, never eliminates, hallucination
- 5 Retrieved is not verified — provenance, not proof

*A good system knows when to say "I don't know." · See docs/RETRIEVAL.md*